

CreaBits Tutor - Documento de Modelo de Datos

Diseño de tablas, DDL PostgreSQL, explicación técnica e inserciones de ejemplo

Versión propuesta para MVP - PostgreSQL / Supabase

1. Arquitectura de datos propuesta

Flujo recomendado: React/Vite -> Backend FastAPI o Node.js -> PostgreSQL/Supabase -> API LLM. El frontend no debería escribir directamente en tablas sensibles; el backend valida código, sesión, asentimiento, ejercicio, conversación y llamadas al LLM.

Capa	Responsabilidad
Frontend	Pantallas de bienvenida, asentimiento, selección de cartas, chat y feedback.
Backend	Validar código anónimo, crear sesiones, guardar mensajes, llamar al LLM y controlar exportaciones.
PostgreSQL/Supabase	Guardar datos estructurados, relaciones, eventos, prompts, mensajes, feedback y auditoría.
LLM	Generar respuestas de Bit usando el system prompt del ejercicio y contexto permitido.

2. Orden recomendado de creación

Orden	Tabla	Motivo
1	grupos	Debe existir antes que las tablas que la referencian.
2	usuarios_app	Debe existir antes que las tablas que la referencian.
3	Estudiantes	Debe existir antes que las tablas que la referencian.
4	consentimientos	Debe existir antes que las tablas que la referencian.
5	categorias_ejercicio	Debe existir antes que las tablas que la referencian.
6	exercises	Debe existir antes que las tablas que la referencian.
7	versiones_ejercicio	Debe existir antes que las tablas que la referencian.
8	categorias_bloque	Debe existir antes que las tablas que la referencian.
9	bloques_scratch	Debe existir antes que las tablas que la referencian.
10	sesiones	Debe existir antes que las tablas que la referencian.
11	conversaciones	Tabla transaccional o de auditoría que depende de las maestras.
12	mensajes	Tabla transaccional o de auditoría que depende de las maestras.
13	eventos	Tabla transaccional o de auditoría que depende de las maestras.
14	feedback_sesion	Tabla transaccional o de auditoría que depende de las maestras.
15	exportaciones	Tabla transaccional o de auditoría que depende de las maestras.
16	auditoria	Tabla transaccional o de auditoría que depende de las maestras.

3. Configuración inicial de PostgreSQL

Antes de crear las tablas, se habilita la extensión pgcrypto para usar gen_random_uuid(). Esto genera UUIDs directamente desde PostgreSQL.

```
-- =====
-- CreaBits Tutor - Script base PostgreSQL
-- Motor recomendado: PostgreSQL 15+ / Supabase PostgreSQL
-- =====

CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Tip: ejecutar este script en Supabase SQL Editor o psql.
-- Primero se crean tablas maestras, luego tablas transaccionales.
```

1. Tabla grupos

¿Para qué sirve esta tabla?

Guarda los grupos o cohortes del piloto. Sirve para separar a los niños por clase, paralelo, sesión sabatina o institución sin mezclar sus datos.

CreaBits Tutor - Modelo de Datos PostgreSQL

Query para construir la tabla

```
CREATE TABLE grupos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nombre TEXT NOT NULL,  
  descripcion TEXT,  
  fecha_inicio DATE,  
  fecha_fin DATE,  
  activo BOOLEAN NOT NULL DEFAULT true,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_grupos_activo ON grupos(activo);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador técnico único. UUID evita exponer números secuenciales y funciona bien en Supabase/PostgreSQL.
nombre	TEXT	NOT NULL	Nombre visible del grupo, por ejemplo CreaBits Sabado Grupo A. TEXT porque no se requiere un largo fijo.
descripcion	TEXT	NULL	Descripción opcional del grupo o contexto.
fecha_inicio	DATE	NULL	Fecha de inicio del grupo. DATE basta porque no requiere hora.
fecha_fin	DATE	NULL	Fecha de cierre del grupo o piloto.
activo	BOOLEAN	DEFAULT true	Permite ocultar grupos antiguos sin borrar historial.
creado_en	TIMESTAMPTZ	DEFAULT now()	Auditoría básica de cuándo fue creado el grupo. TIMESTAMPTZ guarda zona horaria.

PK, FK e índices: por qué ayudan

PK en id para relacionar niños, sesiones y usuarios docentes con un grupo.

Índice en activo para listar rápidamente grupos vigentes.

Ejemplo de inserción

```
INSERT INTO grupos (nombre, descripcion, fecha_inicio, fecha_fin)  
VALUES ('CreaBits Sabado - Grupo A', 'Piloto con niños de 8 a 10 años', '2026-08-08', '2026-08-29')  
RETURNING id;
```

Cómo se simularía en la plataforma

Al iniciar el piloto se crea un grupo por cohorte. Luego cada niño y cada sesión pueden quedar asociados a este grupo, lo que permite analizar conversaciones por clase o cohorte.

2. Tabla usuarios_app

¿Para qué sirve esta tabla?

Guarda a los adultos que administran, investigan o dan soporte al sistema. Los niños no van aquí; los niños entran por código anónimo.

Query para construir la tabla

```
CREATE TABLE usuarios_app (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email TEXT UNIQUE NOT NULL,  
  nombre_completo TEXT NOT NULL,  
  rol TEXT NOT NULL CHECK (rol IN ('researcher','teacher','technical_admin','super_admin')),  
  grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,  
  activo BOOLEAN NOT NULL DEFAULT true,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now(),  
  ultimo_acceso_en TIMESTAMPTZ  
);  
  
CREATE INDEX idx_usuarios_app_rol ON usuarios_app(rol);
```

CreaBits Tutor - Modelo de Datos PostgreSQL

CREATE INDEX idx_usuarios_app_grupo_id ON usuarios_app(grupo_id);

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador interno del usuario adulto.
email	TEXT	UNIQUE NOT NULL	Correo para login de adultos. UNIQUE evita duplicados.
nombre_completo	TEXT	NOT NULL	Nombre del investigador, docente o técnico.
rol	TEXT	CHECK	Controla permisos funcionales: researcher, teacher, technical_admin o super_admin.
grupo_id	UUID	FK NULL	Permite asociar un docente a un grupo específico. ON DELETE SET NULL conserva el usuario si se borra/desactiva el grupo.
activo	BOOLEAN	DEFAULT true	Permite desactivar cuentas sin borrar auditoría.
creado_en	TIMESTAMP TZ	DEFAULT now()	Fecha y hora de creación.
ultimo_acceso_en	TIMESTAMP TZ	NULL	Último ingreso para auditoría.

PK, FK e índices: por qué ayudan

- PK en id para permisos y auditoría.
- UNIQUE en email para impedir dos cuentas con el mismo correo.
- FK grupo_id para vincular docentes con cohortes.
- Índices por rol y grupo para filtros administrativos.

Ejemplo de inserción

```
INSERT INTO usuarios_app (email, nombre_completo, rol, grupo_id)
VALUES ('investigadora@espol.edu.ec', 'Investigadora Responsable', 'researcher', NULL);
```

Cómo se simularía en la plataforma

Cuando un adulto inicia sesión, el backend consulta su rol. Si es researcher puede exportar datos anonimizados; si es teacher puede ver códigos de su grupo y registrar incidentes.

3. Tabla estudiantes

¿Para qué sirve esta tabla?

Guarda la identidad anónima de cada participante. No guarda nombre real, email, foto, voz, IP ni datos identificables. Solo conserva el código público usado en pantalla y metadatos mínimos del estudio.

Query para construir la tabla

```
CREATE TABLE estudiantes(
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  codigo_publico TEXT UNIQUE NOT NULL,
  grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
  edad SMALLINT CHECK (edad BETWEEN 8 AND 10),
  genero_opcion TEXT CHECK (genero_opcion IN ('femenino','masculino','prefiero_no_decir','otro')),
  experiencia_scratch TEXT CHECK (experiencia_scratch IN ('ninguna','un_poco','mucha')),
  experiencia_ia TEXT CHECK (experiencia_ia IN ('ninguna','alguna','frecuente')),
  activo BOOLEAN NOT NULL DEFAULT true,
  creado_en TIMESTAMP TZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_ninos_grupo_id ON ninos(grupo_id);
CREATE INDEX idx_ninos_codigo_publico ON ninos(codigo_publico);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador interno usado por todas las tablas. Evita repetir el código público en cada tabla.
codigo_publico	TEXT	UNIQUE NOT NULL	Alias anónimo visible, por ejemplo tigre-azul-7. UNIQUE evita que dos niños compartan

CreaBits Tutor - Modelo de Datos PostgreSQL

			código.
grupo_id	UUID	FK NULL	Asocia el niño a su grupo. ON DELETE SET NULL protege historial si el grupo se elimina.
edad	SMALLINT	CHECK 8-10	Edad del rango del estudio. SMALLINT porque es número pequeño.
genero_opcion	TEXT	CHECK	Dato opcional de encuesta pre-piloto con valores controlados.
experiencia_scratch	TEXT	CHECK	Experiencia previa declarada con Scratch.
experiencia_ia	TEXT	CHECK	Experiencia previa con asistentes de IA.
activo	BOOLEAN	DEFAULT true	Permite retirar al participante sin borrar inmediatamente el historial hasta aplicar política de datos.
creado_en	TIMESTAMPTZ	DEFAULT now()	Fecha de registro del código anónimo.

PK, FK e índices: por qué ayudan

PK id para relaciones rápidas con sesiones y mensajes.

UNIQUE codigo_publico para login anónimo.

FK grupo_id para análisis por cohorte.

Índices en grupo_id y codigo_publico para búsquedas frecuentes.

Ejemplo de inserción

```
INSERT INTO ninos (codigo_publico, grupo_id, edad, genero_opcion, experiencia_scratch, experiencia_ia)
VALUES ('tigre-azul-7', '00000000-0000-0000-0000-000000000001', 9, 'prefiero_no_decir', 'un_poco', 'ninguna')
RETURNING id;
```

Cómo se simularía en la plataforma

El niño entra usando tigre-azul-7. El backend busca ese código, obtiene el id interno y desde ese momento usa ninos.id para sesiones, conversaciones y mensajes.

5. Tabla categorias_ejercicio

¿Para qué sirve esta tabla?

Guarda las categorías de cartas del prototipo: Animaciones, Juegos, Historias y otras futuras. Permite que el frontend pinte tarjetas y filtros sin quemar todo en código.

Query para construir la tabla

```
CREATE TABLE categorias_ejercicio (
  id TEXT PRIMARY KEY,
  nombre TEXT NOT NULL,
  icono TEXT,
  color_hex TEXT,
  descripcion TEXT,
  orden SMALLINT NOT NULL DEFAULT 0,
  activo BOOLEAN NOT NULL DEFAULT true
);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	TEXT	PK	Identificador estable del JSON: animaciones, juegos, historias. TEXT porque viene como clave semántica.
nombre	TEXT	NOT NULL	Nombre visible en la interfaz.
icono	TEXT	NULL	Emoji o icono asociado a la categoría.
color_hex	TEXT	NULL	Color usado por la UI, por ejemplo #E91E63.
descripcion	TEXT	NULL	Texto corto para la tarjeta o filtro.
orden	SMALLINT	DEFAULT 0	Orden de aparición en pantalla. SMALLINT es suficiente.
activo	BOOLEAN	DEFAULT true	Permite ocultar categorías sin borrarlas.

PK, FK e índices: por qué ayudan

PK id porque ejercicios apuntan a esta categoría y porque el id ya es estable en el JSON.

Ejemplo de inserción

```
INSERT INTO categorias_ejercicio (id, nombre, icono, color_hex, descripcion, orden)
VALUES ('juegos', 'Juegos', '🎮', '#7EC242', 'Crea juegos divertidos', 2);
```

Cómo se simularía en la plataforma

Cuando el frontend carga la pantalla de selección, puede pedir las categorías activas y mostrar filtros como Animaciones, Juegos e Historias.

6. Tabla ejercicios

¿Para qué sirve esta tabla?

Guarda las cartas o actividades principales que el niño puede tocar. Incluye ejercicio guiado y proyecto libre.

Query para construir la tabla

```
CREATE TABLE ejercicios (
  id TEXT PRIMARY KEY,
  categoria_id TEXT REFERENCES categorias_ejercicio(id) ON DELETE SET NULL,
  titulo TEXT NOT NULL,
  icono TEXT,
  descripcion_corta TEXT,
  duracion_estimada_min SMALLINT,
  es_proyecto_libre BOOLEAN NOT NULL DEFAULT false,
  activo BOOLEAN NOT NULL DEFAULT true,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_ejercicios_categoria_id ON ejercicios(categoria_id);
CREATE INDEX idx_ejercicios_activo ON ejercicios(activo);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	TEXT	PK	Identificador del ejercicio, por ejemplo ej_003 o proyecto_libre.
categoria_id	TEXT	FK NULL	Relaciona el ejercicio con Animaciones, Juegos o Historias.
titulo	TEXT	NOT NULL	Título visible de la carta.
icono	TEXT	NULL	Emoji o icono de la carta.
descripcion_corta	TEXT	NULL	Resumen para que el niño entienda la actividad.
duracion_estimada_min	SMALLINT	NULL	Duración aproximada. Puede ser NULL en proyecto libre.
es_proyecto_libre	BOOLEAN	DEFAULT false	Distingue ejercicios guiados de modo libre.
activo	BOOLEAN	DEFAULT true	Permite retirar una carta sin perder históricos.
creado_en	TIMESTAMPTZ	DEFAULT now()	Auditoría de creación.

PK, FK e índices: por qué ayudan

PK id para referenciar ejercicios desde conversaciones.

FK categoria_id para filtros y reportes.

Índices por categoría y activo para cargar rápido la pantalla de cartas.

Ejemplo de inserción

```
INSERT INTO ejercicios (id, categoria_id, titulo, icono, descripcion_corta, duracion_estimada_min)
VALUES ('ej_003', 'juegos', 'Atrapa la estrella', '⭐', 'Mueve el personaje para atrapar estrellas', 25);
```

Cómo se simularía en la plataforma

Cuando el niño toca la carta Atrapa la estrella, el frontend envía exercise_id = ej_003. El backend crea una conversación asociada a ese ejercicio.

7. Tabla versiones_ejercicio

¿Para qué sirve esta tabla?

Guarda las versiones del contenido pedagógico y del system prompt. Es clave para investigación porque permite saber qué versión exacta del tutor recibió cada niño.

Query para construir la tabla

```
CREATE TABLE versiones_ejercicio (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ejercicio_id TEXT NOT NULL REFERENCES ejercicios(id) ON DELETE CASCADE,  
  version TEXT NOT NULL,  
  instruccion_nino TEXT NOT NULL,  
  objetivos_pedagogicos JSONB NOT NULL DEFAULT '[]'::jsonb,  
  pistas_progresivas JSONB NOT NULL DEFAULT '[]'::jsonb,  
  criterios_completado JSONB NOT NULL DEFAULT '[]'::jsonb,  
  preguntas_frecuentes_esperadas JSONB NOT NULL DEFAULT '[]'::jsonb,  
  system_prompt TEXT NOT NULL,  
  activo BOOLEAN NOT NULL DEFAULT true,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (ejercicio_id, version)  
);  
  
CREATE INDEX idx_versiones_ejercicio_activa ON versiones_ejercicio(ejercicio_id, activo);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador de la versión concreta del ejercicio.
ejercicio_id	TEXT	FK NOT NULL	Une la versión con su ejercicio.
version	TEXT	UNIQUE por ejercicio	Etiqueta como v1, v2 o 2026_05.
instruccion_nino	TEXT	NOT NULL	Primer mensaje o instrucción que ve el niño.
objetivos_pedagogicos	JSONB	DEFAULT []	Lista flexible de objetivos; JSONB evita crear una tabla extra para un arreglo simple.
pistas_progresivas	JSONB	DEFAULT []	Pistas por niveles para el tutor.
criterios_completado	JSONB	DEFAULT []	Criterios que definen si la actividad está lograda.
preguntas_frecuentes_esperadas	JSONB	DEFAULT []	Consultas esperadas para análisis posterior.
system_prompt	TEXT	NOT NULL	Prompt que controla el comportamiento de Bit.
activo	BOOLEAN	DEFAULT true	Indica cuál versión se usa por defecto.
creado_en	TIMESTAMPTZ	DEFAULT now()	Auditoría de creación.

PK, FK e índices: por qué ayudan

UNIQUE (ejercicio_id, version) impide duplicar versiones.

FK con ON DELETE CASCADE elimina versiones si se elimina un ejercicio de configuración.

Índice por ejercicio_id y activo para encontrar la versión vigente rápidamente.

Ejemplo de inserción

```
INSERT INTO versiones_ejercicio (  
  ejercicio_id, version, instruccion_nino, objetivos_pedagogicos, pistas_progresivas,  
  criterios_completado, preguntas_frecuentes_esperadas, system_prompt  
) VALUES (  
  'ej_003',  
  'v1',  
  'Vamos a crear un juego: tu personaje tiene que atrapar estrellas moviéndose con las flechas del teclado.',  
  '["Controlar un personaje con el teclado", "Detectar colisiones", "Usar variables para contar puntos"]'::jsonb,  
  '["Busca Eventos", "Mira Sensores", "Crea una variable"]'::jsonb,  
  '["Se mueve con flechas", "Detecta estrella", "Suma puntos"]'::jsonb,  
  '["¿cómo lo muevo con el teclado?", "¿cómo cuento los puntos?"]'::jsonb,  
  'Eres Bit, tutor de Scratch para niños 8-10. NUNCA des la solución completa.'  
)
```

Cómo se simularía en la plataforma

Cuando se genera una respuesta del tutor, el backend guarda version_ejercicio_id o prompt_version en mensajes. Así se puede comparar si una versión del prompt produjo mejores interacciones que otra.

8. Tabla categorias_bloque

¿Para qué sirve esta tabla?

Guarda categorías del catálogo de bloques de Scratch: movimiento, control, eventos, apariencia, variables, operadores, sensores y sonido.

Query para construir la tabla

```
CREATE TABLE categorias_bloque (  
  id TEXT PRIMARY KEY,  
  nombre TEXT NOT NULL,  
  descripcion TEXT,  
  color_hex TEXT,  
  activo BOOLEAN NOT NULL DEFAULT true  
);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	TEXT	PK	Clave estable de la categoría: movimiento, control, eventos, etc.
nombre	TEXT	NOT NULL	Nombre visible.
descripcion	TEXT	NULL	Explica para qué sirve la categoría.
color_hex	TEXT	NULL	Color referencial de Scratch/UI.
activo	BOOLEAN	DEFAULT true	Permite ocultar categorías no usadas.

PK, FK e índices: por qué ayudan

PK id para que los bloques apunten a su categoría.

Ejemplo de inserción

```
INSERT INTO categorias_bloque (id, nombre, descripcion)  
VALUES ('variables', 'Variables', 'Bloques para guardar y cambiar valores como puntos o vidas');
```

Cómo se simularía en la plataforma

El backend puede usar esta tabla para enriquecer la respuesta de Bit cuando el niño pregunte por una categoría concreta de Scratch.

9. Tabla bloques_scratch

¿Para qué sirve esta tabla?

Guarda el catálogo de bloques que Bit puede usar como contexto. Ayuda a mapear preguntas naturales del niño con bloques concretos de Scratch.

Query para construir la tabla

```
CREATE TABLE bloques_scratch (  
  id TEXT PRIMARY KEY,  
  categoria_id TEXT NOT NULL REFERENCES categorias_bloque(id) ON DELETE CASCADE,  
  nombre TEXT NOT NULL,  
  texto_bloque TEXT NOT NULL,  
  imagen_path TEXT,  
  tipo_bloque TEXT NOT NULL CHECK (tipo_bloque IN ('sombrero','apilable','reportador','booleano','contenedor','boton_ui')),  
  parametros JSONB NOT NULL DEFAULT '[]'::jsonb,  
  descripcion TEXT NOT NULL,
```

CreaBits Tutor - Modelo de Datos PostgreSQL

```
sinonimos_naturales JSONB NOT NULL DEFAULT '[]'::jsonb,
ejemplo_uso TEXT,
nivel_dificultad TEXT CHECK (nivel_dificultad IN ('basico','intermedio','avanzado')),
activo BOOLEAN NOT NULL DEFAULT true
);

CREATE INDEX idx_bloques_scratch_categoria ON bloques_scratch(categoria_id);
CREATE INDEX idx_bloques_scratch_nivel ON bloques_scratch(nivel_dificultad);
CREATE INDEX idx_bloques_scratch_sinonimos ON bloques_scratch USING GIN (sinonimos_naturales);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	TEXT	PK	Identificador del bloque del JSON, por ejemplo variables_cambiar.
categoria_id	TEXT	FK NOT NULL	Categoría del bloque.
nombre	TEXT	NOT NULL	Nombre amigable.
texto_bloque	TEXT	NOT NULL	Texto exacto del bloque, por ejemplo cambiar __ en __.
imagen_path	TEXT	NULL	Ruta de imagen si se quiere mostrar visualmente.
tipo_bloque	TEXT	CHECK	Controla tipo de bloque: sombrero, apilable, reportador, etc.
parametros	JSONB	DEFAULT []	Parámetros variables del bloque. JSONB porque cada bloque tiene parámetros distintos.
descripcion	TEXT	NOT NULL	Explicación del bloque.
sinonimos_naturales	JSONB	DEFAULT []	Palabras que el niño puede usar: sumar, avanzar, moverse, etc.
ejemplo_uso	TEXT	NULL	Ejemplo que Bit puede convertir en pista.
nivel_dificultad	TEXT	CHECK	Ayuda a adaptar la respuesta al nivel.
activo	BOOLEAN	DEFAULT true	Permite desactivar bloques no usados.

PK, FK e índices: por qué ayudan

GIN sobre sinonimos_naturales acelera búsquedas dentro del JSONB.

Índices por categoría y dificultad para recuperar contexto relevante.

FK categoría mantiene catálogo consistente.

Ejemplo de inserción

```
INSERT INTO bloques_scratch (
  id, categoria_id, nombre, texto_bloque, tipo_bloque, parametros, descripcion,
  sinonimos_naturales, ejemplo_uso, nivel_dificultad
) VALUES (
  'variables_cambiar', 'variables', 'Cambiar variable', 'cambiar __ en __', 'apilable',
  '[{"nombre":"variable","tipo":"dropdown"}, {"nombre":"valor","tipo":"numero"}]'::jsonb,
  'Suma el valor indicado al valor actual de la variable.',
  '["sumar", "incrementar", "subir puntaje", "restar"]'::jsonb,
  'Si toca estrella, cambiar puntos en 1.',
  'intermedio'
);
```

Cómo se simularía en la plataforma

Si el niño escribe: no sé cómo sumar puntos, el backend puede buscar sinónimos como sumar o puntaje y recuperar el bloque variables_cambiar para pasarlo como contexto al LLM.

10. Tabla sesiones

¿Para qué sirve esta tabla?

Guarda cada entrada del niño a la plataforma. Una sesión equivale a una jornada o uso continuo: ingreso, asentimiento, selección de cartas, chat y despedida.

Query para construir la tabla

```
CREATE TABLE sesiones (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nino_id UUID NOT NULL REFERENCES ninos(id) ON DELETE CASCADE,
  grupo_id UUID REFERENCES grupos(id) ON DELETE SET NULL,
  inicio_en TIMESTAMPTZ NOT NULL DEFAULT now(),
```

CreaBits Tutor - Modelo de Datos PostgreSQL

```
    fin_en TIMESTAMPTZ,
    asentimiento_aceptado BOOLEAN NOT NULL DEFAULT false,
    dispositivo_tipo TEXT,
    estado TEXT NOT NULL DEFAULT 'activa' CHECK (estado IN ('activa','cerrada','abandonada','anulada')),
    metadata JSONB NOT NULL DEFAULT '{} '::jsonb
);

CREATE INDEX idx_sesiones_nino_id ON sesiones(nino_id);
CREATE INDEX idx_sesiones_grupo_id ON sesiones(grupo_id);
CREATE INDEX idx_sesiones_inicio_en ON sesiones(inicio_en);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador de la sesión.
nino_id	UUID	FK NOT NULL	Indica qué niño anónimo usó la plataforma.
grupo_id	UUID	FK NULL	Permite análisis por grupo sin consultar siempre niños.
inicio_en	TIMESTAMPTZ	DEFAULT now()	Marca inicio exacto de la sesión.
fin_en	TIMESTAMPTZ	NULL	Marca cierre. NULL mientras la sesión está activa.
asentimiento_aceptado	BOOLEAN	DEFAULT false	Evita iniciar conversación si no aceptó.
dispositivo_tipo	TEXT	NULL	Opcional: tablet, laptop, desktop. No guardar IP.
estado	TEXT	CHECK	Controla ciclo de vida: activa, cerrada, abandonada o anulada.
metadata	JSONB	DEFAULT {}	Datos flexibles no sensibles: navegador general, número de pantalla, flags técnicos.

PK, FK e índices: por qué ayudan

FK nino_id con ON DELETE CASCADE para cumplir eliminación de datos del participante.

Índice por nino_id para ver historial longitudinal.

Índice por inicio_en para reportes por fecha.

Ejemplo de inserción

```
INSERT INTO sesiones (nino_id, grupo_id, asentimiento_aceptado, dispositivo_tipo, metadata)
VALUES (
    '11111111-1111-1111-1111-111111111111',
    '00000000-0000-0000-0000-000000000001',
    true,
    'laptop',
    '{"pantalla_inicial":"bienvenida", "version_frontend":"v2"}'::jsonb
)
RETURNING id;
```

Cómo se simularía en la plataforma

Después de validar el código y aceptar el asentimiento, el backend crea una sesión. Al cerrar, actualiza fin_en y estado = cerrada.

11. Tabla conversaciones

¿Para qué sirve esta tabla?

Agrupar varios mensajes dentro del mismo hilo. Una sesión puede tener varias conversaciones si el niño cambia de ejercicio o entra a proyecto libre.

Query para construir la tabla

```
CREATE TABLE conversaciones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sesion_id UUID NOT NULL REFERENCES sesiones(id) ON DELETE CASCADE,
    nino_id UUID NOT NULL REFERENCES ninos(id) ON DELETE CASCADE,
    ejercicio_id TEXT NOT NULL REFERENCES ejercicios(id) ON DELETE RESTRICT,
    version_ejercicio_id UUID REFERENCES versiones_ejercicio(id) ON DELETE SET NULL,
    estado TEXT NOT NULL DEFAULT 'activa' CHECK (estado IN ('activa','cerrada','abandonada')),
    inicio_en TIMESTAMPTZ NOT NULL DEFAULT now(),
    fin_en TIMESTAMPTZ,
    metadata JSONB NOT NULL DEFAULT '{} '::jsonb
);
```

```
);

CREATE INDEX idx_conversaciones_sesion_id ON conversaciones(sesion_id);
CREATE INDEX idx_conversaciones_nino_id ON conversaciones(nino_id);
CREATE INDEX idx_conversaciones_ejercicio_id ON conversaciones(ejercicio_id);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador del hilo de conversación.
sesion_id	UUID	FK NOT NULL	Une la conversación con la sesión.
nino_id	UUID	FK NOT NULL	Redundancia controlada para consultas rápidas por niño.
ejercicio_id	TEXT	FK NOT NULL	Ejercicio o carta de la conversación.
version_ejercicio_id	UUID	FK NULL	Versión exacta del ejercicio/prompt usado.
estado	TEXT	CHECK	Activa, cerrada o abandonada.
inicio_en	TIMESTAMPTZ	DEFAULT now()	Inicio del hilo.
fin_en	TIMESTAMPTZ	NULL	Fin del hilo.
metadata	JSONB	DEFAULT {}	Información flexible: modo libre, número de intento, contexto inicial.

PK, FK e índices: por qué ayudan

PK para que mensajes apunten al hilo.

FK ejercicio_id con RESTRICT evita borrar un ejercicio que tiene conversaciones históricas.

Índices por sesión, niño y ejercicio para reconstrucción y análisis.

Ejemplo de inserción

```
INSERT INTO conversaciones (sesion_id, nino_id, ejercicio_id, version_ejercicio_id, metadata)
VALUES (
  '22222222-2222-2222-2222-222222222222',
  '11111111-1111-1111-1111-111111111111',
  'ej_003',
  '33333333-3333-3333-3333-333333333333',
  '{"origen":"seleccion_carta", "modo":"guiado"}'::jsonb
)
RETURNING id;
```

Cómo se simularía en la plataforma

Cuando el niño toca una carta, se abre una conversación. Todos los mensajes de ese ejercicio usan el mismo conversation_id para conservar el orden del diálogo.

12. Tabla mensajes

¿Para qué sirve esta tabla?

Guarda cada mensaje del niño y cada respuesta de Bit. Es la tabla central de investigación porque permite reconstruir la interacción completa.

Query para construir la tabla

```
CREATE TABLE mensajes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversacion_id UUID NOT NULL REFERENCES conversaciones(id) ON DELETE CASCADE,
  sesion_id UUID NOT NULL REFERENCES sesiones(id) ON DELETE CASCADE,
  nino_id UUID NOT NULL REFERENCES ninos(id) ON DELETE CASCADE,
  rol TEXT NOT NULL CHECK (rol IN ('nino','tutor','sistema')),
  contenido TEXT NOT NULL,
  orden_mensaje INTEGER NOT NULL,
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now(),
  proveedor_llm TEXT,
  modelo_llm TEXT,
  prompt_version TEXT,
  input_tokens INTEGER CHECK (input_tokens IS NULL OR input_tokens >= 0),
```

CreaBits Tutor - Modelo de Datos PostgreSQL

```
output_tokens INTEGER CHECK (output_tokens IS NULL OR output_tokens >= 0),
metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
UNIQUE (conversacion_id, orden_mensaje)
);

CREATE INDEX idx_mensajes_conversacion_orden ON mensajes(conversacion_id, orden_mensaje);
CREATE INDEX idx_mensajes_nino_id ON mensajes(nino_id);
CREATE INDEX idx_mensajes_creado_en ON mensajes(creado_en);
CREATE INDEX idx_mensajes_metadata_gin ON mensajes USING GIN (metadata);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador del mensaje.
conversacion_id	UUID	FK NOT NULL	Permite reconstruir el hilo completo.
sesion_id	UUID	FK NOT NULL	Facilita reportes por sesión sin join extra.
nino_id	UUID	FK NOT NULL	Permite análisis longitudinal del niño anónimo.
rol	TEXT	CHECK	Distingue quién envió: nino, tutor o sistema.
contenido	TEXT	NOT NULL	Texto del mensaje. TEXT porque puede variar en longitud.
orden_mensaje	INTEGER	UNIQUE por conversación	Orden explícito dentro del hilo. INTEGER porque crece secuencialmente.
creado_en	TIMESTAMP TZ	DEFAULT now()	Timestamp real del mensaje.
proveedor_llm	TEXT	NULL	Ejemplo: anthropic. NULL para mensajes del niño.
modelo_llm	TEXT	NULL	Ejemplo: claude-haiku. NULL para mensajes del niño.
prompt_version	TEXT	NULL	Versión del prompt usado.
input_tokens	INTEGER	CHECK >=0	Tokens de entrada usados por el LLM. INTEGER por conteo entero.
output_tokens	INTEGER	CHECK >=0	Tokens de salida generados.
metadata	JSONB	DEFAULT {}	Datos flexibles: intención detectada, latencia, bloques usados, estrategia de respuesta.

PK, FK e índices: por qué ayudan

UNIQUE (conversacion_id, orden_mensaje) garantiza orden sin duplicados.

Índice compuesto conversaci3n+orden acelera lectura del chat.

GIN en metadata permite buscar por claves JSONB como detected_intent o safety_flag.

Ejemplo de inserci3n

```
INSERT INTO mensajes (
  conversacion_id, sesion_id, nino_id, rol, contenido, orden_mensaje,
  prompt_version, metadata
) VALUES (
  '44444444-4444-4444-4444-444444444444',
  '22222222-2222-2222-2222-222222222222',
  '11111111-1111-1111-1111-111111111111',
  'nino',
  'No sé c3mo hacer que la estrella sume puntos',
  1,
  'ej_003_v1',
  '{"detected_intent":"pedir_ayuda_variable", "exercise_id":"ej_003", "safety_flag":false}'::jsonb
);

INSERT INTO mensajes (
  conversacion_id, sesion_id, nino_id, rol, contenido, orden_mensaje,
  proveedor_llm, modelo_llm, prompt_version, input_tokens, output_tokens, metadata
) VALUES (
  '44444444-4444-4444-4444-444444444444',
  '22222222-2222-2222-2222-222222222222',
  '11111111-1111-1111-1111-111111111111',
  'tutor',
  '¡Buena pregunta! Para contar puntos necesitas una variable. ¿Recuerdas d3nde se crean las variables en Scratch?',
  2,
  'anthropic', 'claude-haiku', 'ej_003_v1', 420, 38,
  '{"response_strategy":"pregunta_socratica", "hint_level":1, "used_context_blocks":["variables_cambiar]}'::jsonb
);
```

Cada vez que el niño envía un texto, primero se inserta el mensaje del niño. Luego el backend llama al LLM, recibe la respuesta y guarda el mensaje del tutor con tokens, modelo y metadata técnica.

13. Tabla eventos

¿Para qué sirve esta tabla?

Registra acciones que no son mensajes: cambio de ejercicio, aceptación de asentimiento, abandono, completado autoreportado o incidente.

Query para construir la tabla

```
CREATE TABLE eventos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nino_id UUID REFERENCES ninos(id) ON DELETE CASCADE,  
  sesion_id UUID REFERENCES sesiones(id) ON DELETE CASCADE,  
  conversacion_id UUID REFERENCES conversaciones(id) ON DELETE SET NULL,  
  tipo_evento TEXT NOT NULL,  
  payload JSONB NOT NULL DEFAULT '{}':::jsonb,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_eventos_tipo_evento ON eventos(tipo_evento);  
CREATE INDEX idx_eventos_sesion_id ON eventos(sesion_id);  
CREATE INDEX idx_eventos_creado_en ON eventos(creado_en);  
CREATE INDEX idx_eventos_payload_gin ON eventos USING GIN (payload);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador del evento.
nino_id	UUID	FK NULL	Niño asociado. Puede ser NULL para evento global técnico.
sesion_id	UUID	FK NULL	Sesión asociada.
conversacion_id	UUID	FK NULL	Conversación asociada si aplica.
tipo_evento	TEXT	NOT NULL	Tipo: exercise_selected, session_closed, risk_flag, etc.
payload	JSONB	DEFAULT {}	Datos variables del evento: ejercicio anterior, ejercicio nuevo, motivo, severidad.
creado_en	TIMESTAMPTZ	DEFAULT now()	Momento exacto del evento.

PK, FK e índices: por qué ayudan

Índice por tipo_evento para reportes de abandono, cambios o incidentes.

GIN en payload para buscar eventos por propiedades internas.

FKs mantienen trazabilidad con niño, sesión y conversación.

Ejemplo de inserción

```
INSERT INTO eventos (nino_id, sesion_id, conversacion_id, tipo_evento, payload)  
VALUES (  
  '11111111-1111-1111-1111-111111111111',  
  '22222222-2222-2222-2222-222222222222',  
  '44444444-4444-4444-4444-444444444444',  
  'exercise_selected',  
  '{"exercise_id":"ej_003", "exercise_title":"Atrapa la estrella"}':::jsonb  
);
```

Cómo se simularía en la plataforma

Cuando el niño selecciona una carta, cambia de ejercicio o cierra la sesión, se guarda un evento. Esto permite analizar uso aunque el niño no escriba mensajes.

14. Tabla feedback_sesion

¿Para qué sirve esta tabla?

Guarda la micro-encuesta final de cada sesión: cómo se sintió el niño, si Bit ayudó, si algo no se entendió y si le gustaría usarlo otra vez.

Query para construir la tabla

```
CREATE TABLE feedback_sesion (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  session_id UUID NOT NULL REFERENCES sesiones(id) ON DELETE CASCADE,  
  nino_id UUID NOT NULL REFERENCES ninos(id) ON DELETE CASCADE,  
  estado_animo TEXT CHECK (estado_animo IN ('feliz','neutral','triste')),  
  bit_ayudo TEXT CHECK (bit_ayudo IN ('si','un_poco','no')),  
  hubo_algo_no_entendido BOOLEAN,  
  texto_no_entendido TEXT,  
  usaria_otra_vez TEXT CHECK (usaria_otra_vez IN ('si','no_estoy_seguro','no')),  
  comentario_extra TEXT,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (sesion_id)  
);  
  
CREATE INDEX idx_feedback_nino_id ON feedback_sesion(nino_id);  
CREATE INDEX idx_feedback_estado_animo ON feedback_sesion(estado_animo);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador del feedback.
session_id	UUID	FK NOT NULL UNIQUE	Una encuesta final por sesión.
nino_id	UUID	FK NOT NULL	Niño anónimo que respondió.
estado_animo	TEXT	CHECK	Respuesta pictográfica: feliz, neutral o triste.
bit_ayudo	TEXT	CHECK	Medida simple de utilidad percibida.
hubo algo no entendido	BOOLEAN	NULL	Indica si algo de Bit no fue claro.
texto no entendido	TEXT	NULL	Texto opcional.
usaria otra vez	TEXT	CHECK	Intención de uso futuro.
comentario extra	TEXT	NULL	Campo libre para comentarios finales.
creado_en	TIMESTAMPTZ	DEFAULT now()	Fecha de respuesta.

PK, FK e índices: por qué ayudan

UNIQUE session_id evita dos encuestas para la misma sesión.

Índices por nino_id y estado_animo para análisis descriptivo.

Ejemplo de inserción

```
INSERT INTO feedback_sesion (  
  session_id, nino_id, estado_animo, bit_ayudo, hubo_algo_no_entendido,  
  texto_no_entendido, usaria_otra_vez, comentario_extra  
) VALUES (  
  '22222222-2222-2222-2222-222222222222',  
  '11111111-1111-1111-1111-111111111111',  
  'feliz', 'si', false, NULL, 'si', 'Me gustó hacer el juego de la estrella'  
);
```

Cómo se simularía en la plataforma

Al finalizar la clase, la pantalla de despedida envía estas respuestas. La investigadora luego puede cruzar feedback con conversaciones y ejercicios.

15. Tabla exportaciones

¿Para qué sirve esta tabla?

Audita cada exportación JSON o CSV de datos para investigación. No guarda el archivo completo, sino quién exportó, qué filtro aplicó y cuándo.

Query para construir la tabla

```
CREATE TABLE exportaciones (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  solicitado_por UUID REFERENCES usuarios_app(id) ON DELETE SET NULL,  
  formato TEXT NOT NULL CHECK (formato IN ('json','csv')),  
  alcance TEXT NOT NULL CHECK (alcance IN ('sesiones','mensajes','feedback','eventos','dataset_completo')),  
  filtros JSONB NOT NULL DEFAULT '{}':::jsonb,  
  total_registros INTEGER CHECK (total_registros IS NULL OR total_registros >= 0),  
  archivo_path TEXT,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_exportaciones_solicitado_por ON exportaciones(solicitado_por);  
CREATE INDEX idx_exportaciones_creado_en ON exportaciones(creado_en);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador de exportación.
solicitado_por	UUID	FK NULL	Usuario adulto que exportó datos.
formato	TEXT	CHECK	Formato permitido: json o csv.
alcance	TEXT	CHECK	Qué conjunto se exportó.
filtros	JSONB	DEFAULT {}	Filtros usados: fecha, grupo, ejercicio, versión.
total_registros	INTEGER	CHECK >=0	Cantidad exportada.
archivo_path	TEXT	NULL	Ruta interna o nombre de archivo generado, si aplica.
creado_en	TIMESTAMPTZ	DEFAULT now()	Fecha de exportación.

PK, FK e índices: por qué ayudan

FK solicitado_por permite auditoría.

Índice por fecha para revisar exportaciones recientes.

JSONB en filtros permite registrar combinaciones variables.

Ejemplo de inserción

```
INSERT INTO exportaciones (solicitado_por, formato, alcance, filtros, total_registros, archivo_path)  
VALUES (  
  '55555555-5555-5555-5555-555555555555',  
  'csv',  
  'mensajes',  
  '{"grupo":"CreaBits Sabado - Grupo A", "ejercicio_id":"ej_003"}':::jsonb,  
  128,  
  'exports/mensajes_grupo_a_ej003_2026-09-01.csv'  
);
```

Cómo se simularía en la plataforma

Cuando la investigadora exporta datos anonimizados, el sistema deja registro para saber qué se sacó, cuándo y con qué criterios.

16. Tabla auditoria

¿Para qué sirve esta tabla?

Guarda acciones administrativas sensibles: creación de códigos, edición de ejercicios, exportación, cierre de sesión, desactivación o eliminación solicitada.

Documento técnico generado para MVP de investigación

Query para construir la tabla

```
CREATE TABLE auditoria (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  usuario_id UUID REFERENCES usuarios_app(id) ON DELETE SET NULL,  
  accion TEXT NOT NULL,  
  entidad TEXT NOT NULL,  
  entidad_id TEXT,  
  detalle JSONB NOT NULL DEFAULT '{}':::jsonb,  
  creado_en TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE INDEX idx_auditoria_usuario_id ON auditoria(usuario_id);  
CREATE INDEX idx_auditoria_accion ON auditoria(accion);  
CREATE INDEX idx_auditoria_entidad ON auditoria(entidad, entidad_id);
```

Explicación campo por campo

Campo	Tipo	Restricción	Por qué se usa así
id	UUID	PK	Identificador del log.
usuario_id	UUID	FK NULL	Adulto que ejecutó la acción. NULL si fue proceso automático.
accion	TEXT	NOT NULL	Acción realizada: create, update, export, deactivate, delete request.
entidad	TEXT	NOT NULL	Tabla o módulo afectado.
entidad_id	TEXT	NULL	ID de la entidad afectada. TEXT para soportar UUID o IDs tipo ej_003.
detalle	JSONB	DEFAULT {}	Antes/después, motivo, filtros o contexto.
creado_en	TIMESTAMPTZ	DEFAULT now()	Momento de la acción.

PK, FK e índices: por qué ayudan

Índices por usuario, acción y entidad permiten auditoría rápida.

JSONB detalle guarda variaciones sin alterar la tabla.

Ejemplo de inserción

```
INSERT INTO auditoria (usuario_id, accion, entidad, entidad_id, detalle)  
VALUES (  
  '55555555-5555-5555-5555-555555555555',  
  'export',  
  'mensajes',  
  NULL,  
  '{"formato":"csv", "motivo":"analisis_preliminar", "anonimizado":true}':::jsonb  
);
```

Cómo se simularía en la plataforma

Cada vez que un adulto exporta, cambia configuración o procesa una solicitud de eliminación, se registra aquí. Esto fortalece trazabilidad y manejo responsable de datos.

Consultas útiles para validar la simulación

Reconstruir una conversación completa

```
SELECT m.orden_mensaje, m.rol, m.contenido, m.creado_en  
FROM mensajes m  
WHERE m.conversacion_id = '44444444-4444-4444-4444-444444444444'  
ORDER BY m.orden_mensaje ASC;
```

Ver mensajes por ejercicio

```
SELECT e.titulo, m.rol, m.contenido, m.creado_en  
FROM mensajes m  
JOIN conversaciones c ON c.id = m.conversacion_id  
JOIN ejercicios e ON e.id = c.ejercicio_id  
WHERE e.id = 'ej_003'
```

```
ORDER BY m.creado_en;
```

Contar mensajes por niño anónimo

```
SELECT n.codigo_publico, COUNT(*) AS total_mensajes
FROM mensajes m
JOIN ninos n ON n.id = m.nino_id
GROUP BY n.codigo_publico
ORDER BY total_mensajes DESC;
```

Buscar mensajes donde metadata detectó intención de pedir ayuda con variables

```
SELECT contenido, metadata->>'detected_intent' AS intencion
FROM mensajes
WHERE metadata->>'detected_intent' = 'pedir_ayuda_variable';
```

Apéndice A. Script completo de creación

El script completo también se entrega como archivo .sql separado. A continuación se deja una referencia:

script_creacion_base_creabits_tutor.sql

Apéndice B. Reglas de diseño aplicadas

UUID como PK: evita exponer secuencias y facilita integración con Supabase.

TEXT para campos variables: PostgreSQL maneja TEXT eficientemente y evita límites artificiales.

TIMESTAMPZ para fechas con hora: conserva la zona horaria y permite análisis temporal correcto.

JSONB para metadatos: útil cuando los datos pueden cambiar entre versiones del experimento.

CHECK constraints: evitan valores inválidos como roles no permitidos o estados inexistentes.

FKs: protegen integridad referencial entre niños, sesiones, conversaciones y mensajes.

Índices: aceleran búsquedas frecuentes, como reconstruir chats, filtrar por ejercicio o auditar exportaciones.

ON DELETE CASCADE en datos de niños: permite eliminar datos asociados si existe una solicitud válida de retiro/eliminación.

ON DELETE SET NULL en usuarios o grupos: conserva historial aunque se desactive una cuenta o grupo.